CSC331: Data Structures - BMCC Fall 2025
Professor Byron
Program #2 specifications: Linked list
Due: 1159pm Fri 10-10-2025
No credit received for late submission

Your task for this assignment is to implement a linked list data structure in C++.

1. Implement a transaction-based linked list data structure using a C++ object. The program will be interactive. A transaction will be entered at the command line after a short prompt and output will be displayed on the console. Note: a batch of input transactions in a plain text file can be processed using redirection.

2. A linked list of book IDs, titles and quantities will be created and updated using Add, Remove, Order, Sell and List transactions. Assume that all input transactions are formatted correctly. Any integer book ID is valid. Any string book title is valid. Any integer book quantity is valid. Transaction types will be processed as follows:
    - Add – To add a book to the linked list, process a transaction in the form of "A", space, ID, space, title. For example: "A 10 book1". If the new book ID is not on the linked list, the book ID and title will be added to the end of the linked list with quantity set to zero. If the book ID is already on the list, the transaction will fail. One of two messages will be displayed: (1) "book added" or (2) "book not added".
    - Remove – To remove a book from the linked list, process a transaction in the form of "R", space, ID. For example: "R 10". If the book ID is on the list, the book will be removed from the linked list. If the book ID is not on the list, the transaction will fail. One of two messages will be displayed: (1) "book removed" or (2) "book not removed".
    - List – To display the books in a numbered list, enter a transaction in the form of "L". Each book (ID, title and quantity) in the linked list will be displayed next to a sequential number. Each book will be on a line by itself.
    - Order – To order a quantity of books, process a transaction in the form of "O", space, ID, space, quantity. For example: "O 10 7". If the book ID is not on the linked list, the transaction will fail. If the book ID is on the list, the quantity for the book will be increased by the quantity amount on the transaction. One of two messages will be displayed: (1) "order added" or (2) "order not added".
    - Sell – To sell a quantity of books, process a transaction in the form of "S", space, ID, space, quantity. For example: "S 10 7". If the book ID is not on the linked list, the transaction will fail. If the book ID is on the list, the quantity amount for the

book will be decreased by the quantity amount on the transaction. One of two messages will be displayed: (1) “books sold” or (2) “books not sold”.

- Quit – To terminate the program, enter a transaction in the form of “Q”.

In this assignment, use the partially completed class called book_f25.h. Use an include statement in your main program to obtain the book_f25.h header file. The following methods are already working: addBook and list. Your assignment is to add the methods removeBook, orderBooks and sellBooks to the book class in the book_f25.h header file. No other changes are permitted in the book class in the book_f25.h header file.

3. Here is sample dialogue of the running program:

```
$ prog2
enter transaction: A 10 book1
book added
enter transaction: A 10 book2
book not added
enter transaction: A 20 book3
book added
enter transaction: L
1.    10 book1 0
2.    20 book3 0
enter transaction: R 30
book not removed
enter transaction: R 10
book removed
enter transaction: O 10 7
books not ordered
enter transaction: O 20 7
books ordered
enter transaction: L
1.    20 book3 7
enter transaction: Q
$
```

A sample plain text file of transactions, such as prog2.dat, may contain the following:

```
A 10 book1
A 10 book2
A 20 book3
L
R 30
R 10
O 10 7
```

O 20 7
L
Q

Using redirection, the program will be run as follows:

$ prog2 < prog2.dat

The output produced will be similar to the output shown above with redirection.

4. Your C++ program file should be named csc331_section_prog2_lastname.cpp. Your program should contain comments starting on line 1 of the program containing the course ID and section, your full name, the program file name, the program due date and the program purpose. Programmers are always encouraged to add additional comments throughout the program that might be helpful to the reader of your source code.

5. Submit your C++ program source file (i.e., cpp file) and modified book_f25.h header file as separate, unzipped attachments to an email message to kbyron@bmcc.cuny.edu using an email subject in this form: "csc331_section_prog2_lastname". A link to a file located on a remote server will not be accepted. If you submit your assignment at least one week early, it will be graded. If you wish, you may then re-submit your complete assignment prior to the due date for a potentially higher score.

6. Do your own work. All students submitting copies of the same program will receive grades of zero for the assignment and may be referred to the Dean of Students, as described in the BMCC plagiarism policy.

7. Grading rubric

   Partial or extra credit will be awarded as follows:

   0%: compiling errors
   50%: significant logic with no compiling errors
   70%: add, list and quit commands working
   90%: add, list, quit and remove commands working
   100%: add, list, quit, remove and order commands working
   110%: all commands working